UNSUPERVISED LEARNING: STRUCTURE & PATTERNS

Clustering, Dimensionality Reduction, and Anomaly Detection

Dr. Lumumba W. Victor

Center for Data Analytics and Modeling
Chuka University

24th March 2026

Resource: `https://cdam.chuka.ac.ke`

Learning Objectives (Part 1)

By the end of this session, participants will be able to:

1. Understand the unsupervised learning framework and use cases
2. Explain the mathematical foundations of clustering algorithms
3. Implement K-Means and Hierarchical Clustering in R
4. Evaluate cluster quality using internal metrics

Learning Objectives (Part 2)

5. Apply dimensionality reduction techniques (PCA, t-SNE, UMAP)
6. Understand Gaussian Mixture Models and EM algorithm
7. Detect anomalies using isolation-based methods
8. Extract association rules from transactional data

Session Outline (Part 1)

1. Unsupervised Learning Framework
2. Distance & Similarity Metrics
3. Data Preprocessing for Unsupervised
4. K-Means Clustering
5. K-Means: Mathematical Foundation
6. Choosing Optimal K
7. K-Means: R Implementation
8. Hierarchical Clustering
9. Linkage Methods
10. Dendrograms Interpretation

Session Outline (Part 2)

11. Hierarchical: R Implementation
12. DBSCAN: Density-Based Clustering
13. DBSCAN: Parameters & Math
14. Gaussian Mixture Models (GMM)
15. EM Algorithm Overview
16. GMM: R Implementation
17. Principal Component Analysis (PCA)
18. PCA: Mathematical Foundation
19. Variance Explained & Scree Plot
20. PCA: R Implementation

Session Outline (Part 3)

21. Non-Linear Reduction (t-SNE, UMAP)
22. Factor Analysis Overview
23. Anomaly Detection Concepts
24. Isolation Forest & LOF
25. Association Rules (Apriori)
26. Rule Metrics (Support, Confidence)
27. Cluster Validation Metrics
28. Stability & Robustness
29. Best Practices Summary
30. Resources & Q&A

What is Unsupervised Learning?

Definition: Learn patterns from *unlabeled* data X without target variable y.

Main Tasks:

- **Clustering:** Group similar observations (e.g., customer segmentation)
- **Dimensionality Reduction:** Compress features while retaining info (e.g., PCA)
- **Anomaly Detection:** Identify rare/outlier instances (e.g., fraud)
- **Association Rules:** Discover co-occurrence patterns (e.g., market basket)

Key Challenge: No ground truth to validate performance directly.

Unsupervised Learning Workflow

1. Data Collection
2. Preprocessing & Scaling
3. Dimensionality Reduction (Optional)
4. Model Training
5. Internal Validation
6. Interpretation

Iterative: Refine based on cluster interpretability and stability.

Distance Metrics (Part 1)

Crucial for Clustering & KNN:

- **Euclidean:** $d(x, y) = \sqrt{\sum_{j=1}^{p}(x_j - y_j)^2}$
  - Default for continuous data; sensitive to scale
- **Manhattan:** $d(x, y) = \sum_{j=1}^{p} |x_j - y_j|$
  - Robust to outliers; grid-like distance
- **Minkowski:** $d(x, y) = (\sum_{j=1}^{p} |x_j - y_j|^q)^{1/q}$
  - Generalization (Euclidean $q = 2$, Manhattan $q = 1$)

Rule: Always scale features before computing distances.

Distance Metrics (Part 2)

For Categorical & Mixed Data:

- **Hamming:** Proportion of mismatched coordinates
  - For binary/categorical vectors
- **Gower:** Handles mixed numeric + categorical
  - Weighted average of feature-wise distances
- **Cosine Similarity:** $\text{sim}(x, y) = \frac{x \cdot y}{\|x\|\|y\|}$
  - Measures angle, not magnitude; good for text

Choice matters: Different metrics yield different cluster structures.

Preprocessing for Unsupervised

Critical Steps:

- **Scaling:** Standardize (mean=0, sd=1) for distance-based methods
- **Missing Values:** Impute carefully (distance undefined for NA)
- **Outliers:** Detect/remove (can distort centroids in K-Means)
- **Encoding:** One-hot for categorical (increases dimensionality)

R Code Snippet:

```
Check distribution summary(data_scaled)Shouldbeapproxmean = 0, sd = 1
```

Warning: K-Means and PCA are sensitive to feature scales.

Curse of Dimensionality

Problem: As dimensions p increase, data becomes sparse.

Consequences:

- Distance metrics become less meaningful (all points equidistant)
- Clustering algorithms struggle to find structure
- Overfitting risk increases

Solutions:

- Feature selection (remove irrelevant variables)
- Dimensionality reduction (PCA, t-SNE)
- Increase sample size n

Rule of Thumb: If $p \approx n$, consider reduction before clustering.

K-Means: Conceptual Overview

Goal: Partition n observations into K clusters.

Intuition:

- Each cluster represented by its centroid (mean)
- Points assigned to nearest centroid
- Iteratively update centroids until convergence

Assumptions:

- Clusters are spherical and similar in size
- Features are continuous and scaled

Limitation: Requires specifying K in advance.

K-Means: Algorithm Steps

1. Initialize K centroids (randomly or k-means++)
2. **Assignment Step:** Assign each point to nearest centroid
3. **Update Step:** Recompute centroids as mean of assigned points
4. Repeat steps 2-3 until centroids stabilize (convergence)

Points assign to nearest centroid

Convergence: Guaranteed to local optimum (not necessarily global).

K-Means: Mathematical Foundation

Objective Function (Within-Cluster Sum of Squares):

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k is the centroid of cluster k.

Optimization:

- Non-convex problem
- Solved via coordinate descent (Lloyd's algorithm)
- Sensitive to initialization (use multiple starts)

Complexity: $O(n \cdot K \cdot I \cdot p)$ where I = iterations.

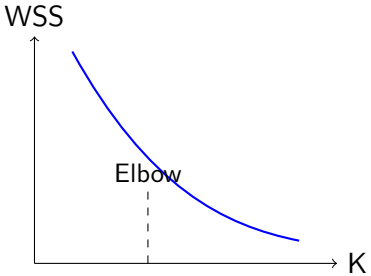
Choosing Optimal K

Methods:

- **Elbow Method:** Plot WSS vs. K; look for "elbow" point
- **Silhouette Score:** Measure how similar point is to own cluster vs. others

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

- **Gap Statistic:** Compare WSS to null reference distribution



K-Means: R Implementation

```r
library(cluster)
library(factoextra)

# Scale data
data_scaled <- scale(iris[, -5])

# Determine optimal K
fviz_nbclust(data_scaled, kmeans, method = "wss")

# Run K-Means (K=3)
set.seed(123)
km_res <- kmeans(data_scaled, centers = 3,
                 nstart = 25)

# Visualize clusters
fviz_cluster(km_res, data = data_scaled)

# View cluster sizes
km_res$size
```

Tip: Use `nstart > 1` to avoid poor local optima.

K-Means: Strengths and Limitations

Strengths:

- Simple, fast, scalable to large n
- Guaranteed convergence
- Works well when clusters are spherical

Limitations:

- Requires specifying K
- Sensitive to outliers and initialization
- Assumes equal variance and spherical shapes
- Struggles with non-convex clusters

Use Case: Customer segmentation, image compression, initial exploration.

Hierarchical Clustering: Concept

Goal: Build a hierarchy of clusters (tree structure).

Two Approaches:

- **Agglomerative (Bottom-Up):**
  - Start: each point is a cluster
  - Merge closest pairs iteratively
- **Divisive (Top-Down):**
  - Start: all points in one cluster
  - Split iteratively (less common)

Output: Dendrogram (visualizes merge history).

Linkage Methods

How to measure distance between clusters?

- **Single:** Min distance between any two points
  - Prone to chaining effect
- **Complete:** Max distance between any two points
  - Produces compact clusters
- **Average:** Average distance between all pairs
  - Balanced approach
- **Ward's:** Minimize increase in total within-cluster variance
  - Similar to K-Means objective

Recommendation: Ward's method often performs best for continuous data.

Dendrograms Interpretation

Reading the Tree:

- **Y-axis:** Distance at which clusters merged
- **Cut Height:** Horizontal line determines number of clusters
- **Branch Length:** Longer = more dissimilar clusters

Tip: Look for large vertical gaps to choose cut height.

Hierarchical: Mathematical Foundation

Distance Matrix Update (Lance-Williams Formula):

$$d(C_i \cup C_j, C_k) = \alpha_i d(C_i, C_k) + \alpha_j d(C_j, C_k) + \beta d(C_i, C_j) + \gamma |d(C_i, C_k) - d(C_j, C_k)|$$

Coefficients depend on linkage:

- **Ward:** Minimizes variance increase
- **Average:** $\alpha_i = \frac{n_i}{n_i + n_j}, \beta = 0, \gamma = 0$

Complexity: $O(n^3)$ or $O(n^2 \log n)$; not scalable to very large n.

Hierarchical: R Implementation

```r
# Compute distance matrix
dist_mat <- dist(data_scaled, method = "euclidean")

# Hierarchical clustering (Ward's method)
hc_res <- hclust(dist_mat, method = "ward.D2")

# Plot dendrogram
plot(hc_res, hang = -1, cex = 0.6)

# Cut tree to get K clusters
clusters <- cutree(hc_res, k = 3)

# Visualize with factoextra
fviz_dend(hc_res, k = 3,
          rect = TRUE,        # Draw rectangles around clusters
          show_labels = FALSE)
```

Tip: Use `rect.hclust()` to highlight clusters on base plot.

Hierarchical: Strengths and Limitations

Strengths:

- No need to specify K in advance
- Dendrogram provides rich visualization
- Works with any distance metric
- Deterministic (same result every run)

Limitations:

- Computationally expensive ($O(n^2)$ to $O(n^3)$)
- Sensitive to noise and outliers
- Once merged, cannot be undone (greedy)
- Difficult to interpret for very large n

Use Case: Small to medium datasets, phylogenetic trees, taxonomy.

DBSCAN: Conceptual Overview

Density-Based Spatial Clustering of Applications with Noise.

Core Idea: Clusters are dense regions separated by sparse areas.

Point Types:

- **Core Point:** Has $\geq$ MinPts within radius Eps
- **Border Point:** Within Eps of core but $<$ MinPts neighbors
- **Noise Point:** Neither core nor border (outlier)

Advantage: Can find arbitrary shaped clusters; identifies outliers.

DBSCAN: Parameters

Two Key Hyperparameters:

- **Eps (ϵ):** Neighborhood radius
  - Too small: many points labeled noise
  - Too large: clusters merge
- **MinPts:** Minimum points to form dense region
  - Rule of thumb: $\geq$ dimensionality $+ 1$
  - Typical: 4 for 2D, 10+ for high dimensions

Selection: Use K-distance plot (elbow method on sorted distances).
Benefit: Does not require specifying number of clusters.

DBSCAN: Mathematical Foundation

Definitions:

- **Eps-neighborhood:** $N_\epsilon(p) = \{q \in D \mid \text{dist}(p, q) \leq \epsilon\}$
- **Directly Density-Reachable:** $q \in N_\epsilon(p)$ and $|N_\epsilon(p)| \geq \text{MinPts}$
- **Density-Connected:** Path of density-reachable points exists

Cluster Definition:

1. Maximality: All density-reachable points included
2. Connectivity: Any two points in cluster are density-connected

Complexity: $O(n \log n)$ with spatial indexing (e.g., KD-tree).

DBSCAN: R Implementation

```r
library(dbscan)

# Determine Eps using k-distance plot
kNNdistplot(data_scaled, k = 4)
# Look for elbow ~0.5

# Run DBSCAN
db_res <- dbscan(data_scaled, eps = 0.5, minPts = 4)

# View clusters (0 = noise)
table(db_res$cluster)

# Visualize
plot(db_res, data = data_scaled,
     main = "DBSCAN Clustering")

# Identify noise points
noise_points <- which(db_res$cluster == 0)
```

Note: Cluster 0 represents noise/outliers.

DBSCAN: Strengths and Limitations

Strengths:

- No need to specify K
- Detects arbitrary shaped clusters
- Robust to outliers (identifies noise)
- Efficient for spatial data

Limitations:

- Sensitive to Eps and MinPts tuning
- Struggles with varying density clusters
- Performance degrades in high dimensions
- Border points assignment can be ambiguous

Use Case: Geographic data, anomaly detection, non-spherical clusters.

GMM: Conceptual Overview

Concept: Data generated from a mixture of K Gaussian distributions.

Probabilistic Model:

$$P(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$$

where π_k = mixing coefficient ($\sum \pi_k = 1$).

Difference from K-Means:

- Soft assignment (probabilities) vs. hard assignment
- Accounts for cluster covariance (elliptical shapes)

Flexibility: Can model clusters with different sizes and orientations.

EM Algorithm Overview

Expectation-Maximization (EM) Algorithm:

1. **Initialize:** Guess parameters $\theta = \{\pi_k, \mu_k, \Sigma_k\}$
2. **E-Step:** Compute responsibility γ_{nk} (prob point n belongs to k)
3. **M-Step:** Update parameters using responsibilities
4. Repeat until log-likelihood converges

Convergence: Guaranteed to increase likelihood; may reach local optimum.

GMM: Mathematical Foundation

Log-Likelihood Function:

$$\ln P(X|\theta) = \sum_{n=1}^{N} \ln \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(x_n|\mu_k, \Sigma_k) \right)$$

Responsibility (E-Step):

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(x_n|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)}$$

Parameter Updates (M-Step):

$$\mu_k^{new} = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_{nk} x_n, \quad \Sigma_k^{new} = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_{nk} (x_n - \mu_k)(x_n - \mu_k)^T$$

Model Selection: Use BIC or AIC to choose K.

GMM: R Implementation

```r
library(mclust)

# Fit GMM with BIC selection
gmm_res <- Mclust(data_scaled)

# Summary (optimal K selected)
summary(gmm_res)

# Plot classification
plot(gmm_res, what = "classification")

# Plot uncertainty
plot(gmm_res, what = "uncertainty")

# Access parameters
gmm_res$parameters$mean     # Mu
gmm_res$parameters$variance # Sigma
```

Advantage: `Mclust` automatically selects optimal K via BIC.

GMM: Strengths and Limitations

Strengths:

- Soft clustering (probabilistic membership)
- Models elliptical clusters (covariance)
- Mathematically grounded (maximum likelihood)
- Provides density estimation

Limitations:

- Sensitive to initialization
- Requires specifying K (unless using BIC)
- Computationally heavier than K-Means
- Assumes Gaussian distribution per cluster

Use Case: Customer segmentation with overlap, density estimation.

PCA: Conceptual Overview

Principal Component Analysis: Linear dimensionality reduction.

Goal: Project data onto orthogonal axes (principal components) that maximize variance.

Key Properties:

- PC1: Direction of maximum variance
- PC2: Orthogonal to PC1, next max variance
- PCs are uncorrelated linear combinations of original features

Use: Visualization, noise reduction, multicollinearity handling.

PCA: Mathematical Foundation

Eigenvalue Decomposition:

1. Compute covariance matrix $\Sigma = \frac{1}{n-1} X^T X$
2. Solve eigenproblem: $\Sigma v = \lambda v$
3. Eigenvectors v = Principal Components
4. Eigenvalues λ = Variance explained

Projection:

$$Z = XV$$

where V contains top k eigenvectors.

Alternative: Singular Value Decomposition (SVD) on centered X.

Variance Explained & Scree Plot

Proportion of Variance Explained (PVE):

$$\text{PVE}_j = \frac{\lambda_j}{\sum_{i=1}^{p} \lambda_i}$$

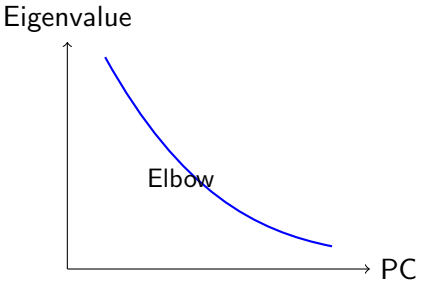
Choosing Components:

- **Scree Plot:** Look for elbow in eigenvalues
- **Cumulative Variance:** Retain enough PCs to explain 80-95%
- **Kaiser Rule:** Keep PCs with eigenvalue > 1 (if scaled)

Variance Explained & Scree Plot

Proportion of Variance Explained (PVE):

$$\text{PVE}_j = \frac{\lambda_j}{\sum_{i=1}^{p} \lambda_i}$$



PCA: R Implementation

```r
library(stats)
library(factoextra)

# Run PCA (scale = TRUE is critical)
pca_res <- prcomp(data_scaled, center = TRUE, scale. = TRUE)

# Summary (variance explained)
summary(pca_res)

# Scree plot
fviz_eig(pca_res, addlabels = TRUE)

# Biplot (variables + observations)
fviz_pca_biplot(pca_res,
                label = "var",       # Show variable names
                habillage = "none") # No clustering
```

Interpretation: Arrows show contribution of original features to PCs.

PCA: Interpretation

Loadings: Correlation between original variables and PCs.

- High absolute loading = variable important for that PC
- Sign indicates direction of relationship

Scores: Coordinates of observations in PC space.

- Used for visualization (2D/3D plots)
- Used as input for clustering/regression

Caution: PCs are linear combinations; interpretability may decrease.

PCA: Strengths and Limitations

Strengths:

- Reduces dimensionality and noise
- Removes multicollinearity
- Computationally efficient (linear algebra)
- Visualization of high-dimensional data

Limitations:

- Linear method (misses non-linear structure)
- Sensitive to outliers
- Components may be hard to interpret
- Requires scaling for meaningful results

Use Case: Preprocessing for supervised learning, exploratory visualization.

t-SNE: Conceptual Overview

t-Distributed Stochastic Neighbor Embedding.

Goal: Visualize high-dimensional data in 2D/3D.

Key Idea: Preserve local structure (neighborhoods).

- Similar points in high-D $\rightarrow$ close in low-D
- Dissimilar points $\rightarrow$ far apart

Note: Non-linear; good for visualization, not feature extraction.

t-SNE: Mathematical Foundation

Probability Distribution (High-D):

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2/2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2/2\sigma_i^2)}$$

Probability Distribution (Low-D): Student-t distribution (heavy tails)

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l}(1 + \|y_k - y_l\|^2)^{-1}}$$

Optimization: Minimize KL Divergence $KL(P||Q)$ via gradient descent.
Perplexity: Controls balance between local and global structure.

UMAP: Conceptual Overview

Uniform Manifold Approximation and Projection.

Advantages over t-SNE:

- Faster computation
- Preserves more global structure
- Scalable to larger datasets
- Can be used for general dimensionality reduction (not just viz)

Theory: Based on Riemannian geometry and algebraic topology.
Parameters: `n_neighbors` (local vs global), `min_dist` (cluster tightness).

Non-Linear Reduction: R Implementation

```r
library(Rtsne)
library(umap)

# t-SNE
set.seed(123)
tsne_res <- Rtsne(data_scaled, dims = 2, perplexity = 30)
plot(tsne_res$Y, col = clusters, pch = 19)

# UMAP
umap_res <- umap(data_scaled, n_neighbors = 15, min_dist = 0.1)
plot(umap_res$layout, col = clusters, pch = 19)

# Compare preservation
# UMAP often shows clearer cluster separation
```

Tip: Run multiple times; stochastic algorithms may vary.

Visualization Best Practices

For PCA/t-SNE/UMAP Plots:

- Color by known labels (if available) to validate
- Color by cluster assignment to inspect grouping
- Label axes with variance explained (PCA only)
- Use transparency for dense regions
- Avoid over-interpreting distances in t-SNE

Warning: t-SNE/UMAP distances are not metric; focus on clusters.

Factor Analysis: Concept

Goal: Explain correlations among variables using latent factors.

Model:

$$X = \Lambda F + \epsilon$$

where F = latent factors, Λ = loadings, ϵ = error.

Difference from PCA:

- PCA: Variance maximization (data reduction)
- FA: Covariance modeling (latent structure discovery)

Use Case: Psychology (intelligence), social sciences, survey validation.

Factor Analysis: Math

Covariance Structure:

$$\Sigma = \Lambda\Lambda^T + \Psi$$

where Ψ is diagonal matrix of unique variances.

Estimation:

- Principal Axis Factoring
- Maximum Likelihood

Rotation:

- **Varimax (Orthogonal):** Factors uncorrelated
- **Promax (Oblique):** Factors allowed to correlate

Goal of Rotation: Simplify structure (make loadings 0 or high).

Factor Analysis: R Implementation

```r
library(psych)

# Determine number of factors
fa.parallel(data_scaled, fa = "fa")

# Extract factors (Maximum Likelihood)
fa_res <- fa(data_scaled, nfactors = 3,
             rotate = "varimax", fm = "ml")

# View loadings
print(fa_res$loadings, cutoff = 0.3)

# Factor scores
scores <- fa_res$scores
```

Interpretation: High loading = variable strongly associated with factor.

FA vs. PCA Summary

Feature	PCA
Goal	Data compression
Assumption	None (algebraic)
Variance	Explains total variance
Components	Orthogonal linear combos
Use Case — Preprocessing, viz — Psychology, survey validation	

Rule: Use PCA for prediction prep; use FA for theory building.

Anomaly Detection: Concepts

Goal: Identify rare items/events differing from majority.

Types of Anomalies:

- **Point Anomalies:** Single instance abnormal (e.g., high transaction)
- **Contextual Anomalies:** Abnormal in specific context (e.g., high temp in winter)
- **Collective Anomalies:** Collection of instances abnormal together

Challenge: Highly imbalanced data; often no labeled anomalies.

Isolation Forest & LOF

Isolation Forest:

- Randomly split data; anomalies isolated faster
- Shorter path length = higher anomaly score
- Efficient ($O(n)$)

Local Outlier Factor (LOF):

- Density-based
- Compares local density of point to neighbors
- LOF $\gg 1$ indicates outlier

Use: Fraud detection, network intrusion, system health monitoring.

Anomaly Detection: R Implementation

```r
library(IsolationForest)
library(RLOF)

# Isolation Forest
iforest_res <- isolationForest(data_scaled)
scores <- predict(iforest_res, data_scaled)
# Lower score = more anomalous

# LOF
lof_scores <- lof(data_scaled, k = 10)
# Higher score = more anomalous

# Identify top 5% outliers
threshold <- quantile(scores, 0.05)
outliers <- which(scores < threshold)
```

Validation: Use precision@k if some labels available.

Association Rules: Concepts

Goal: Discover interesting relations between variables (e.g., Market Basket).

Rule Form: $\{A\} \rightarrow \{B\}$ (If A bought, then B bought).

Key Metrics:

- **Support:** Frequency of itemset $\{A, B\}$
- **Confidence:** $P(B|A)$
- **Lift:** $\frac{P(B|A)}{P(B)}$ (Independence measure)

Lift ¿ 1: Positive association; Lift ¡ 1: Negative association.

Association Rules: R Implementation

```r
library(arules)
library(arulesViz)

# Load transactions
data("Groceries")

# Mine rules (Apriori algorithm)
rules <- apriori(Groceries,
                 parameter = list(supp = 0.01,
                                  conf = 0.5,
                                  target = "rules"))

# Sort by lift
rules_sorted <- sort(rules, by = "lift")

# Visualize
plot(rules_sorted, method = "graph")
inspect(head(rules_sorted, 10))
```

Tip: Adjust support/confidence thresholds to manage rule count.

Internal Validation Metrics

No Ground Truth Required:

- **Silhouette Coefficient:**
  - Range [-1, 1]; higher = better defined clusters
- **Davies-Bouldin Index:**
  - Lower = better (minimize within, maximize between)
- **Calinski-Harabasz Index:**
  - Ratio of between-cluster to within-cluster dispersion

R Package: `cluster::silhouette()`, `fpc::cluster.stats()`.

External Validation Metrics

When Ground Truth Labels Exist:

- **Adjusted Rand Index (ARI):**
  - Corrects for chance; 1 = perfect match
- **Normalized Mutual Information (NMI):**
  - Information shared between clusters and labels
- **Fowlkes-Mallows Index:**
  - Geometric mean of precision and recall

Use: Benchmarking algorithms on labeled datasets (e.g., Iris).

Stability & Robustness

Cluster Stability:

- Run algorithm multiple times (different seeds)
- Measure consistency of assignments (Jaccard similarity)
- Unstable clusters = unreliable structure

Robustness Checks:

- Subsample data and re-cluster
- Add noise and check cluster persistence
- Vary hyperparameters (K, Eps) slightly

Best Practice: Report stability metrics alongside quality metrics.

Best Practices Summary

Before Clustering:

1. Scale features (critical for distance-based)
2. Remove outliers (or use robust methods like DBSCAN)
3. Reduce dimensions if p is very high

During Modeling:

1. Try multiple algorithms (K-Means, Hierarchical, DBSCAN)
2. Tune hyperparameters systematically
3. Validate with multiple metrics

After Modeling:

1. Interpret clusters using domain knowledge
2. Profile clusters (mean values per feature)
3. Document limitations and assumptions

Summary of Models

Model	Type	Best For	Key Param
K-Means	Centroid	Spherical, large n	K
Hierarchical	Tree	Small n, dendrogram	Linkage
DBSCAN	Density	Arbitrary shapes, noise	Eps, MinPts
GMM	Probabilistic	Elliptical, soft assign	K
PCA	Linear Reduction	Preprocessing, viz	Components
t-SNE/UMAP	Non-Linear	Visualization	Perplexity/Neighbors

No Free Lunch: Choose based on data structure and business goal.

Resources & Q&A

Key Takeaways:

- Unsupervised learning discovers hidden structure without labels
- Preprocessing (scaling) is critical for distance-based methods
- Validate clusters using internal metrics (Silhouette, DB Index)
- Combine dimensionality reduction with clustering for high-D data
- Interpret results with domain expertise

Further Learning:

- *An Introduction to Statistical Learning* (Ch. 10, 12)
- *Pattern Recognition and Machine Learning* (Bishop)
- CDAM Resources: `https://cdam.chuka.ac.ke`
- R Packages: `cluster`, `factoextra`, `dbscan`, `mclust`

Thank you! Questions?

Contact: `lumumbavictor172@gmail.com` — CDAM, Chuka University